

## Technology Stack

|                      |               |
|----------------------|---------------|
| <b>Date</b>          | 30 June       |
| <b>Project Name</b>  | Rising Waters |
| <b>Maximum Marks</b> | 2 Marks       |

### Define Your Technology Stack:

Identify the programming languages, frameworks, databases, front-end tools, back-end tools, and APIs that you will use to build your solution. A well-chosen technology stack ensures that your application is scalable, maintainable, and aligned with your project requirements.

Fill out the table below to detail the specific technologies chosen for each component of your architecture and provide a brief justification for your choice.

### Technology Stack Details

| S.No | Architecture Component/<br>Layer                                                 | Technology Chosen                        | Justification / Purpose                                                                                                                 |
|------|----------------------------------------------------------------------------------|------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------|
| 1    | <b>Frontend / Client-Side</b><br>(e.g., React, Angular, HTML/<br>CSS)            | HTML5, CSS3,<br>JavaScript (Vanilla ES6) | Ensures clean rendering of dashboard components without unnecessary heavy client framework dependencies, keeping page load latency low. |
| 2    | <b>Backend / Server-Side</b><br>(e.g., Node.js, Python/Django,<br>Java)          | Python / Flask<br>Framework              | Enables seamless native execution and integration of data processing routines and machine learning models (XGBoost/KNN via Joblib).     |
| 3    | <b>Database / Data Storage</b><br>(e.g., MySQL, MongoDB,<br>PostgreSQL)          | SQLite / Flat CSV Logs                   | Provides lightweight, high-performance structured mapping for regional timeline log sets and model parameter matrices.                  |
| 4    | <b>Cloud / Hosting /<br/>Deployment</b><br>(e.g., AWS, Azure, Heroku,<br>Docker) | Docker Container<br>Platforms            | Ensures fully containerized portable deployment capabilities across dynamic high-availability local hosting nodes.                      |
| 5    | <b>Version Control &amp; CI/CD</b><br>(e.g., GitHub, GitLab, Jenkins)            | GitHub & Git Actions                     | Enables clean collaborative code versioning alongside automated workspace build validations prior to container updates.                 |

| S.No | Architecture Component/<br>Layer                                             | Technology Chosen                     | Justification / Purpose                                                                                               |
|------|------------------------------------------------------------------------------|---------------------------------------|-----------------------------------------------------------------------------------------------------------------------|
| 6    | <b>Third-Party APIs / Other Tools</b><br>(e.g., Stripe, Twilio, Google Maps) | OpenWeatherMap API & Twilio Alert API | Fetches real-time localized rainfall parameters and dispatches instantaneous crisis SMS alerts to field coordinators. |